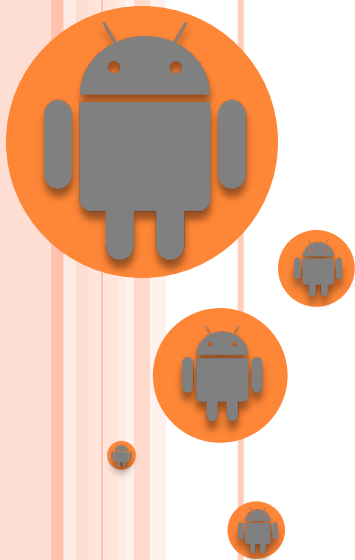
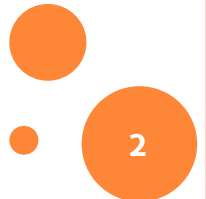


IMÁGENES



IMÁGENES

- Para incorporar una imagen en nuestra aplicación:
 - Sacando una fotografía en ese momento
 - Usando una aplicación de fotografía
 - Desde nuestra propia aplicación
 - Seleccionando una imagen de nuestra galería
- ¿Qué hacer luego con la imagen?
 - Ponerla en un *ImageView* (al cerrar la actividad se perdería)
 - Almacenarla
 - En un fichero
 - En una Base de Datos (Local o remota)
 - En Firebase



IMÁGENES

- Es recomendable indicar en el manifiesto que se requiere tener cámara fotográfica

```
<uses-feature android:name="android.hardware.camera2.full" android:required="true"/>
```

Existen distintos niveles

- Hay que declarar y gestionar los permisos correspondientes para cada caso

Gestionar el hardware (para sacar fotos desde nuestra aplicación)

Leer de la galería

```
<uses-permission android:name="android.permission.CAMERA"/>  
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>  
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

Guardar en la galería (incluye el permiso de lectura)



IMÁGENES

- Sacar una fotografía desde nuestra propia aplicación
 - Usando el paquete *android.hardware.camera2*
 - Hay que gestionar las opciones de la cámara, su apertura, la previsualización de la imagen, etc.

<https://github.com/android/camera-samples>

- Sacar una fotografía usando una aplicación de fotografía o seleccionar una imagen de la galería
 - Funcionan a través de *ActivityResultLauncher*
 - Se gestiona la fotografía en el método *registerForActivityResult(...)*



IMÁGENES

- Seleccionar una imagen de la galería
 - En la actividad, se crea el launcher con

```
private ActivityResultLauncher<PickVisualMediaRequest> pickMedia =
    registerForActivityResult(new
ActivityResultContracts.PickVisualMedia(), uri -> {
    // Callback is invoked after the user selects a media item
    or closes the
    // photo picker.
    if (uri != null) {
        Log.d("PhotoPicker", "Selected URI: " + uri);
        ImageView elImageView = findViewById(R.id.imageView);
        elImageView.setImageURI(uri);
    } else {
        Log.d("PhotoPicker", "No media selected");
    }
});
```

- Recoge la imagen y la pone en un *ImageView*
- Desde donde queramos lanzar la selección de imagen:

```
pickMedia.launch(new PickVisualMediaRequest.Builder()
    .setMediaType(ActivityResultContracts.PickVisualMedia.ImageAndVideo.INSTANCE)
    .build());
```

Permite selección de
imagen o vídeo



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía

- El intent y la llamada al launcher para lanzar la foto

Pedir/verificar
antes el permiso
para usar la cámara

```
Intent elIntentFoto= new Intent(MediaStore.ACTION_IMAGE_CAPTURE);  
takePictureLauncher.launch(elIntentFoto);
```

- Al principio de la actividad, hay que instanciar el Launcher para recoger la imagen y ponerla en un *ImageView*

```
private ActivityResultLauncher<Intent> takePictureLauncher =  
    registerForActivityResult(new  
ActivityResultContracts.StartActivityForResult(), result -> {  
        if (result.getResultCode() == RESULT_OK &&  
result.getData() != null) {  
            Bundle bundle = result.getData().getExtras();  
            ImageView elImageView = findViewById(R.id.imageView);  
            Bitmap laminiatura = (Bitmap) bundle.get("data");  
            elImageView.setImageBitmap(laminiatura);  
        } else {  
            Log.d("TakenPicture", "No photo taken");  
        }  
    });
```



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Si sólo queremos recoger la foto y guardarla en un fichero
 - *Hay que decidir en qué directorio se quiere almacenar*

- *data/data/**nombrepaquete**/files* → Carpeta privada de la app

```
File eldirectorio = this.getFilesDir();
```

- *sdcard/Android/data/**nombrepaquete**/files/Pictures* → Carpeta externa, privada de la app, pero accesible para el usuario

Las carpetas privadas se eliminan con la app

```
File eldirectorio = this.getExternalFilesDir(Environment.DIRECTORY_PICTURES);
```



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Si sólo queremos recoger la foto y guardarla en un fichero
 - Hay que decidir un nombre único para el fichero

```
String timeStamp = new SimpleDateFormat("yyyyMMdd_HH:mm:ss", Locale.getDefault()).format(new Date());  
String nombrefichero = "IMG_" + timeStamp + "_";
```

Es imposible sacar dos fotografías
en el mismo instante



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Si sólo queremos recoger la foto y guardarla en un fichero

```
private ActivityResultLauncher<Intent> takePictureLauncher =
    registerForActivityResult(new ActivityResultContracts.StartActivityForResult(), result -> {
        if (result.getResultCode() == RESULT_OK && result.getData() != null) {
            Bundle bundle = result.getData().getExtras();
            Bitmap laminiatura = (Bitmap) bundle.get("data");
            //GUARDAR COMO FICHERO
            // Memoria externa
            File eldirectorio = this.getExternalFilesDir(Environment.DIRECTORY_PICTURES);

            String timeStamp = new SimpleDateFormat("yyyyMMdd_HH:mm:ss", Locale.getDefault()).format(new Date());
            String nombrefichero = "IMG_" + timeStamp + "_";
            File imagenFich = new File(eldirectorio, nombrefichero + ".jpg");

            OutputStream os;
            try {
                os = new FileOutputStream(imagenFich);
                laminiatura.compress(Bitmap.CompressFormat.JPEG, 100, os);
                os.flush();
                os.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
```

IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Si queremos la imagen en tamaño completo, hay que indicar dónde se va a almacenar. No se puede gestionar una imagen a tamaño completo sin almacenarla.
 - Nuestra aplicación decide dónde almacenar la fotografía
 - La fotografía la almacena la aplicación de fotografía (no la nuestra)
 - Para poder indicarle a la aplicación de fotografía dónde debe almacenar la imagen hay que usar un *Uri* generado a través de un *FileProvider* (desde Android 7, API 24)
 - Genera un Uri de tipo *content* en vez de *file*.

Es lo que permite "compartir" directorios



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - El FileProvider se define mediante XML en la carpeta *res/xml*
 - El elemento raíz es `<paths>`
 - El resto de elementos indican qué directorios se quieren “compartir”
 - `<files-path>` → *this.getFilesDir()*
 - `<external-files-path>` → *this.getExternalFilesDir(null)*
 -

Equivalencias

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
  <external-files-path
    name="externo"
    path="/" />
  <files-path
    name="privado"
    path="/" />
</paths>
```

Indicamos si queremos “compartir” la raíz o algún subdirectorio concreto



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Hay que definir el *FileProvider* en el manifiesto de la aplicación

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.tema16_img_ej1">
    ...
    <application
        ...>
        <provider
            android:authorities="com.example.tema16_img_ej1.provider"
            android:name="androidx.core.content.FileProvider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/configalmacen"/>
            </provider>
        ...
    </application>
</manifest>
```

Un nombre único basado en un dominio. Se suele usar el paquete

De dónde importar el FileProvider.

Si es público

Dar permisos de acceso

Nombre del fichero XML donde está definido el FileProvider



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - El intent

```
String timeStamp = new SimpleDateFormat("yyyyMMdd_HH:mm:ss", Locale.getDefault()).format(new Date());
String nombrefich = "IMG_" + timeStamp + "_";
File directorio=this.getFilesDir();
File fichImg = null;
Uri uriimagen = null;
try {
    fichImg = File.createTempFile(nombrefich, ".jpg",directorio);
    uriimagen = FileProvider.getUriForFile(this, "com.example.tema17ejercicio1.provider", fichImg);
} catch (IOException e) {
    ...
}
Intent elIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
elIntent.putExtra(MediaStore.EXTRA_OUTPUT, uriimagen);
startActivityForResult(elIntent);
```

El directorio donde almacenar la imagen. Debe ser compatible con lo definido en el *FileProvider*

El nombre único definido en el atributo *authorities* del manifiesto

Uri del fichero para almacenar la imagen



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Recoger la imagen y ponerla en un *ImageView*

```
protected void onActivityResult(int requestCode, int resultCode, Intent data) {  
    ...  
    if (requestCode == CODIGO_FOTO_ARCHIVO && resultCode == RESULT_OK) {  
        elImageView.setImageURI(uriimagen);  
    }  
    ...  
}
```

Uri del fichero donde está la imagen



IMÁGENES

- Sacar una fotografía usando una aplicación de fotografía
 - Si nuestra aplicación debe trabajar con varias imágenes, hacerlo en su tamaño y resolución original puede generar problemas de memoria
 - Es aconsejable escalarlas al tamaño que se van a mostrar, pero manteniendo su aspecto (el ratio)

```
int anchoDestino = elImageView.getWidth();
int altoDestino = elImageView.getHeight();
int anchoImagen = bitmapFoto.getWidth();
int altoImagen = bitmapFoto.getHeight();
float ratioImagen = (float) anchoImagen / (float) altoImagen;
float ratioDestino = (float) anchoDestino / (float) altoDestino;
int anchoFinal = anchoDestino;
int altoFinal = altoDestino;
if (ratioDestino > ratioImagen) {
    anchoFinal = (int) ((float) altoDestino * ratioImagen);
} else {
    altoFinal = (int) ((float) anchoDestino / ratioImagen);
}
Bitmap bitmappredimensionado = Bitmap.createScaledBitmap(bitmapFoto, anchoFinal, altoFinal, true);
```

bitmapFoto tiene cargada la imagen en tamaño original

Contiene una imagen escalada al tamaño del destino



IMÁGENES

- En APIs < 29 (Android 10), al almacenar una imagen nueva, la galería y el repositorio multimedia (*MediaStore*) no se actualizan
 - Hay que reiniciar el dispositivo
 - Para evitarlo, al añadir una imagen nueva, se lanza un Broadcast para que se actualice el repositorio multimedia
 - Sólo tiene sentido con las imágenes que se almacenan en carpetas accesibles a otras aplicaciones

```
Intent mediaScanIntent = new Intent(Intent.ACTION_MEDIA_SCANNER_SCAN_FILE);  
Uri contentUri = Uri.fromFile(fichImg);  
mediaScanIntent.setData(contentUri);  
this.sendBroadcast(mediaScanIntent);
```

Objeto File con la imagen

- En la API 29 se hace automáticamente
 - *ACTION_MEDIA_SCANNER_SCAN_FILE* deprecated



IMÁGENES

- Incluir la imagen en una Base de Datos Local
 - Hay que crear un campo de tipo BLOB en la tabla

```
"CREATE TABLE imagenes( identificador TEXT PRIMARY KEY not null,  
                           foto blob not null, titulo TEXT not null)"
```

- Hay que transformar la imagen en un array de bytes

Bitmap con la imagen

```
ByteArrayOutputStream stream = new ByteArrayOutputStream();  
foto.compress(Bitmap.CompressFormat.PNG, 100, stream);  
byte[] fototransformada = stream.toByteArray();
```

- Hay que parametrizar la consulta de inserción

Cada parámetro
con su tipo

```
int identificador=1;  
String titulo="foto de ejemplo";  
String sql = "INSERT INTO imagenes VALUES (?, ?, ?)";  
SQLiteStatement statement = bd.compileStatement(sql);  
statement.clearBindings();  
statement.bindLong(1, identificador);  
statement.bindBlob(2, fototransformada);  
statement.bindString(3, titulo);  
statement.executeInsert();
```

Objeto SQLiteDatabase



IMÁGENES

- Extraer la imagen de una Base de Datos Local
 - Se obtiene un cursor con la información de la BD

```
String sql = "SELECT * FROM imagenes";  
Cursor cursor = db.rawQuery(sql, null);
```

Objeto SQLiteDatabase

- Se extrae la información del cursor

Número de la columna

```
cursor.moveToFirst();  
if (cursor.getCount() > 0) {  
    byte[] image = cursor.getBlob(1);  
    String titulo=cursor.getString(3);  
    int id=cursor.getInt(2);  
    ByteArrayInputStream imageStream = new ByteArrayInputStream(image);  
    Bitmap laimg = BitmapFactory.decodeStream(imageStream);  
}
```

Cada dato con su tipo



IMÁGENES

- Incluir la imagen en una Base de Datos en el Servidor
 - Hay que crear un php que almacene la información

```
<?php
...
$image = $_POST['imagen'];
$id = $_POST['identificador'];
$tit = $_POST['titulo'];
$sql = "INSERT INTO imagenes (identificador,foto,titulo) VALUES (?,?,:)";
$stmt = mysqli_prepare($con,$sql);
mysqli_stmt_bind_param($stmt,"sss",$id,$image,$tit);
mysqli_stmt_execute($stmt);
if (mysqli_stmt_errno($stmt)!=0) {
    echo 'Error de sentencia: ' . mysqli_stmt_error($stmt);
}
...
?>
```

Las variables recibidas

El tipo de cada parámetro. 3 strings

Control de errores



IMÁGENES

- Incluir la imagen en una Base de Datos en el Servidor
 - Hay que convertir el bitmap en un string en Base64

```
ByteArrayOutputStream stream = new ByteArrayOutputStream();  
foto.compress(Bitmap.CompressFormat.PNG, 100, stream);  
byte[] fototransformada = stream.toByteArray();  
String fotoen64 = Base64.encodeToString(fototransformada, Base64.DEFAULT);
```

- Hay que generar los parámetros que se quieren enviar al servicio web

Los valores a enviar

```
Uri.Builder builder = new Uri.Builder()  
    .appendQueryParameter("identificador", pid)  
    .appendQueryParameter("imagen", pimagen)  
    .appendQueryParameter("titulo", ptitulo);  
String parametrosURL = builder.build().getEncodedQuery();
```



IMÁGENES

- Extraer la imagen de una Base de Datos del Servidor
 - Crear un php que extraiga el blob de la BD

```
<?php
...
$query = "SELECT * FROM imagenes where identificador=1";
$result = mysqli_query($con,$query);
if (!$result) {
    echo 'Ha ocurrido algún error: ' . mysqli_error($con);
    exit;
}
else {
    $photo = mysqli_fetch_array($result);
    echo base64_decode($photo['foto']);
}
...
?>
```



IMÁGENES

- Extraer la imagen de una Base de Datos del Servidor
 - Llamar al php , recoger el resultado y transformarlo a un *Bitmap*

```
...
URLConnection conn = (URLConnection) url.openConnection();
int responseCode = 0;
try {
    responseCode = conn.getResponseCode();
    if (responseCode == HttpURLConnection.HTTP_OK) {
        Bitmap elBitmap = BitmapFactory.decodeStream(conn.getInputStream());
    }
} catch (IOException e) {
    ...
}
...
```



IMÁGENES

- Almacenar la imagen en un fichero en el Servidor
 - Hay que crear un php que almacene la imagen

```
<?php
...
$base=$_POST['imagen'];
$binary=base64_decode($base);
header('Content-Type: bitmap; charset=utf-8');
$file = fopen('uploaded_image.jpg', 'w+');
fwrite($file, $binary);
fclose($file);
...
?>
```

Nombre del fichero o path completo

Modo de escritura. Si no existe, lo crea



IMÁGENES

- Almacenar la imagen en un fichero en el Servidor
 - En Android el proceso es igual que cuando se quiere almacenar en la BD
 - Transformar el bitmap a un string en base64
 - Crear los parámetros
 - Pasar los parámetros a formato URL
 - Llamar al php
 - El php deberá además almacenar en la BD, la relación entre el nuevo fichero (con su nombre o su path) y el contenido que se desee



IMÁGENES

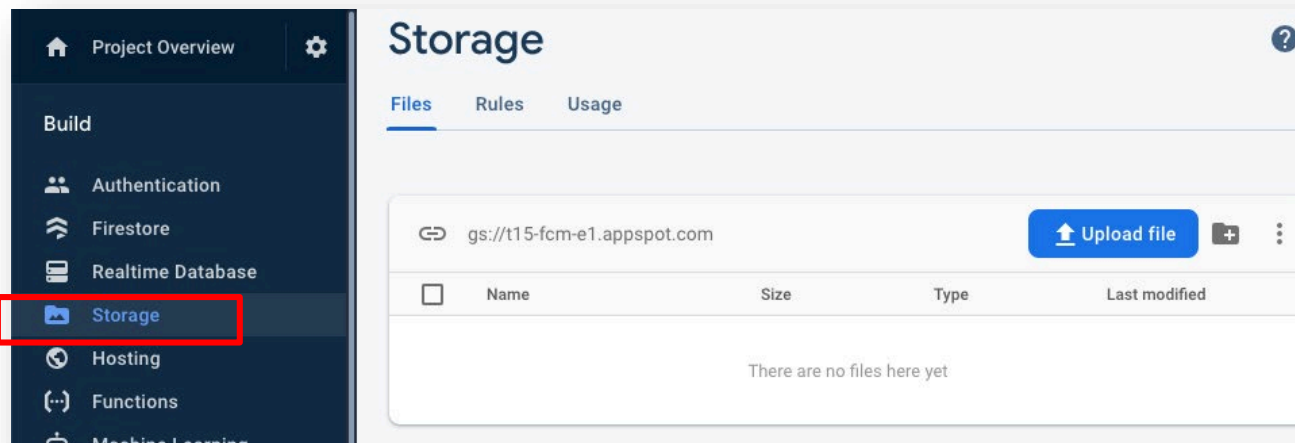
○ Descargar la imagen del Servidor

```
String direccion = "https://servidor/uploaded_image.jpg";
URL destino = new URL(direccion);
...
HttpsURLConnection conn = (HttpsURLConnection) destino.openConnection();
int responseCode = 0;
try {
    responseCode = conn.getResponseCode();
    if (responseCode == HttpsURLConnection.HTTP_OK) {
        Bitmap elBitmap = BitmapFactory.decodeStream(conn.getInputStream());
    }
} catch (IOException e) {
    ...
}
...
```



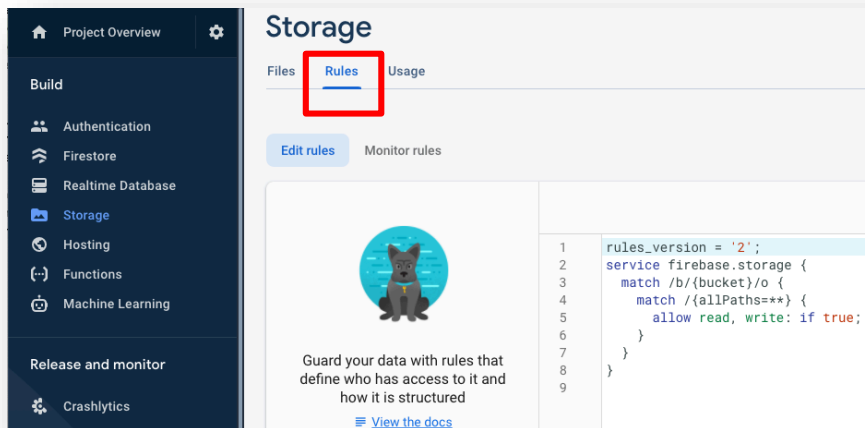
IMÁGENES

- Firebase Storage para almacenar la imagen
 - Firebase ofrece un espacio de almacenamiento para guardar ficheros en la nube
 - Para comenzar, crear un "Bucket" (espacio de almacenamiento) en Firebase
 - Desde la consola Web:



IMÁGENES

- Firebase Storage para almacenar la imagen
 - Modificar los permisos del Bucket para permitir almacenamiento sin autenticación:
 - Sección "Rules", añadir el siguiente código:



```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /{allPaths=**} {
      allow read, write: if true;
    }
  }
}
```



IMÁGENES

- Firebase Storage para almacenar la imagen
 - Conectar la aplicación con Firebase Storage
 - En el menú "Tools" de Android Studio, elegir "Firebase"
 - En el menú contextual
 - 1) Click en "Connect to Firebase" y seguir el asistente
 - 2) Click en "Add Cloud Storage to your app"
 - Añade las dependencias necesarias en Gradle



IMÁGENES

- Firebase Storage para almacenar la imagen
 - Subir la imagen a Firebase

```
FirebaseStorage storage = FirebaseStorage.getInstance();  
storageRef = storage.getReference();  
StorageReference spaceRef = storageRef.child("miImagen.jpg");  
spaceRef.putFile(uriimagen);
```

URI de la imagen en
el dispositivo

Nombre del fichero o path
a usar en Firebase Storage



IMÁGENES

- Firebase Storage para almacenar la imagen
 - Cargar la imagen desde Firebase
 - Incluir librería Glide en el fichero Gradle

```
implementation 'com.github.bumptech.glide:glide:4.7.1'
```

- Desde la actividad:
 - Puede tardar unos segundos en cargar

```
StorageReference pathReference = storageRef.child("space.jpg");  
pathReference.getDownloadUrl().addOnSuccessListener(new OnSuccessListener<Uri>() {  
    @Override  
    public void onSuccess(Uri uri) {  
        Glide.with(getApplicationContext()).load(uri).into(elImageView);  
    }  
});
```

Nombre del
fichero o path en
Firebase Storage

Referencia al
ImageView



IMÁGENES

○ Ejercicio 1

- Crear una aplicación que tenga un *ImageView* y permita al usuario elegir una imagen de la galería o sacar una en ese momento y ponerla en el *ImageView*.
- Añadir a la aplicación la opción de almacenar la Imagen en una base de datos remota y una actividad donde se recupere la imagen de la base de datos.
- Modificar la aplicación para que se guarde y se cargue desde Firebase Storage.

